

**Evolución y Fortalecimiento del Backend de EcoHome Store: Persistencia, Autenticación,
Autorización y Gestión Segura del Catálogo mediante Node.js, Express y PostgreSQL**

Juan Camilo Avila Acosta

Geraldine Moncaleano, Rafael Ricardo Mantilla

Optimización y Desarrollo Avanzado de Aplicaciones Multiplataforma

2026

Actividad 1. Persistencia del catálogo en PostgreSQL

1. Modelo de datos y script SQL

Para esta primera etapa se requieren dos entidades principales: usuarios (users) y productos (products). La tabla users permitirá posteriormente implementar el sistema de autenticación y autorización, mientras que products almacenará de forma permanente el catálogo de EcoHome Store. La utilización de PostgreSQL permite incorporar un sistema de persistencia relacional para almacenar de manera permanente la información del catálogo (PostgreSQL Global Development Group, 2025).

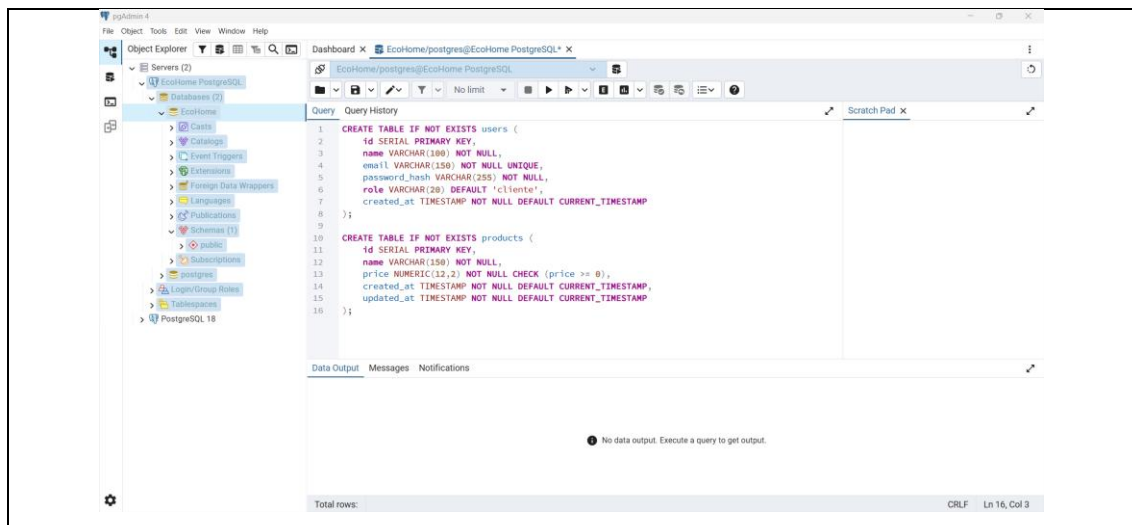


Imagen 1. Creación de tablas. PostgreSQL. Fuente: Propia.

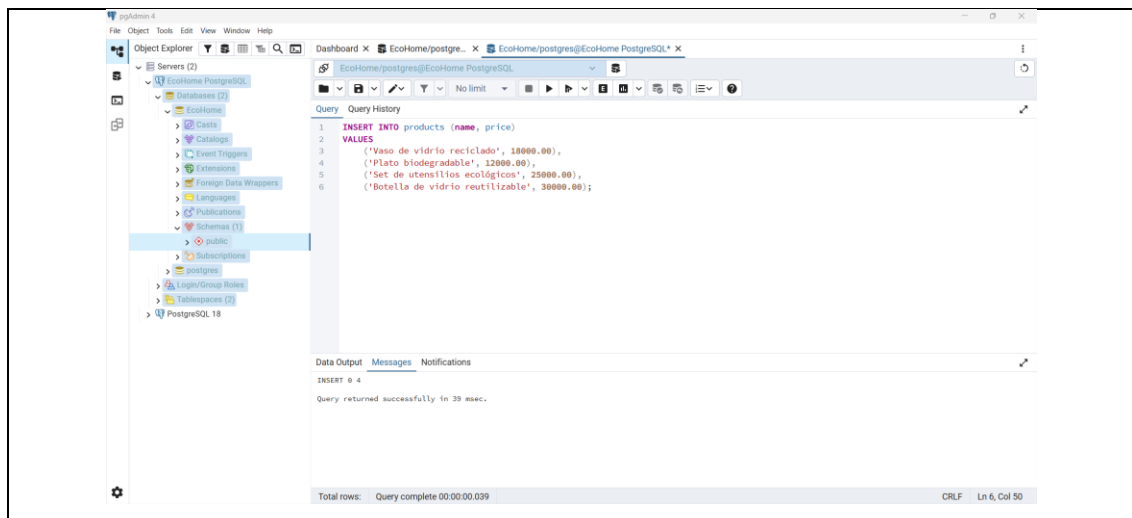
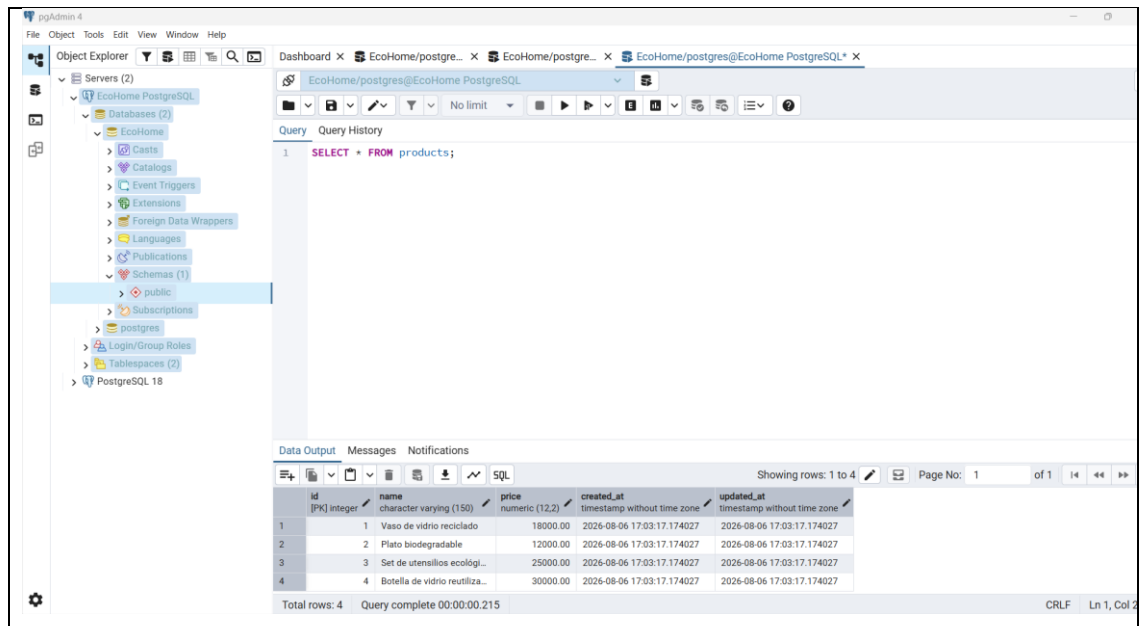


Imagen 2. Integración de datos. PostgreSQL. Fuente: Propia.



The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Servers' tree with 'EcoHome PostgreSQL' selected. The main pane shows the 'Query' editor with the SQL query 'SELECT * FROM products;'. Below the query editor, the 'Data Output' pane displays the results of the query in a table format.

id	name	price	created_at	updated_at
1	Vaso de vidrio reciclado	18000.00	2026-08-06 17:03:17.174027	2026-08-06 17:03:17.174027
2	Plato biodegradable	12000.00	2026-08-06 17:03:17.174027	2026-08-06 17:03:17.174027
3	Set de utensilios ecológi...	25000.00	2026-08-06 17:03:17.174027	2026-08-06 17:03:17.174027
4	Botella de vidrio reutiliza...	30000.00	2026-08-06 17:03:17.174027	2026-08-06 17:03:17.174027

Total rows: 4 Query complete 00:00:00.215

Imagen 3. Consulta sobre la base de datos. PostgreSQL. Fuente: Propia.

La tabla users contiene la información necesaria para que posteriormente se pueda implementar el registro y autenticación. El campo email debe ser único para evitar que diferentes cuentas utilicen las mismas credenciales, mientras que password_hash almacenará únicamente el resultado del proceso de hash de la contraseña y nunca la contraseña en texto plano.

La tabla products reemplaza el arreglo que anteriormente se encontraba en memoria. Los campos created_at y updated_at permiten realizar posteriormente un seguimiento básico de cuándo fue creado o modificado cada producto.

```
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) DEFAULT 'cliente',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    price NUMERIC(12,2) NOT NULL CHECK (price >= 0),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Scrip 1. Creacion de tablas en PostgreSQL. Fuente: Propia.

En este diseño, SERIAL permite generar automáticamente los identificadores de los registros. El precio utiliza NUMERIC(12,2) porque es más apropiado para representar valores monetarios que utilizar tipos de punto flotante.

2. Configuración de conexión a PostgreSQL

Para establecer una conexión segura entre el backend desarrollado con Node.js y la base de datos PostgreSQL, se configuraron variables de entorno mediante un archivo .env. Esta configuración permite separar los parámetros de conexión de la lógica de programación y evita que las credenciales de acceso a la base de datos queden escritas directamente dentro de los archivos JavaScript del proyecto. El uso de JavaScript permite estructurar la lógica del backend y desarrollar las diferentes funcionalidades necesarias para la gestión del catálogo (Flanagan, 2020).

El proyecto fue desarrollado inicialmente en Visual Studio Code y se configuró un archivo .env ubicado en la raíz del proyecto, al mismo nivel que el archivo package.json. En este archivo se definieron los parámetros necesarios para establecer la conexión con la base de datos EcoHome, creada previamente en PostgreSQL 18 mediante pgAdmin.

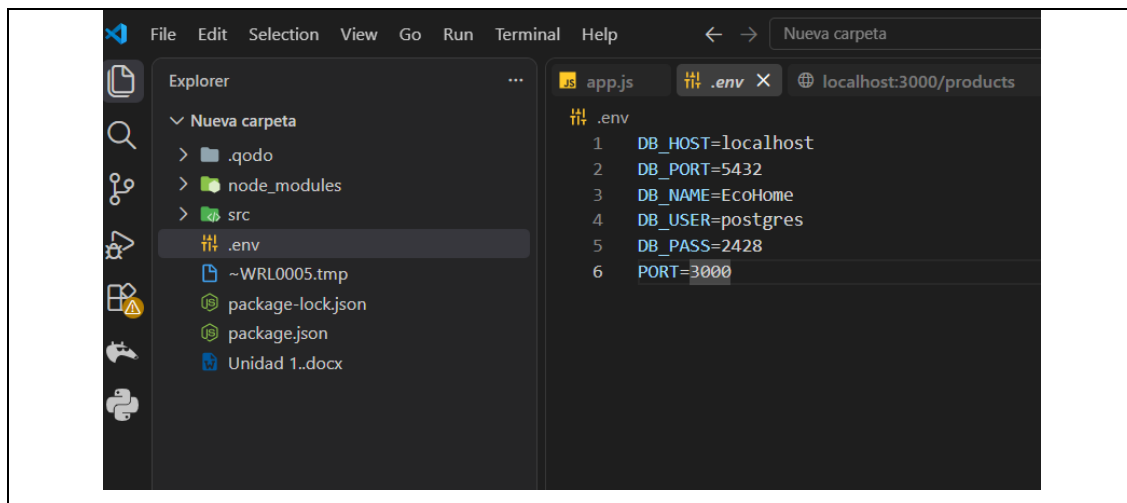
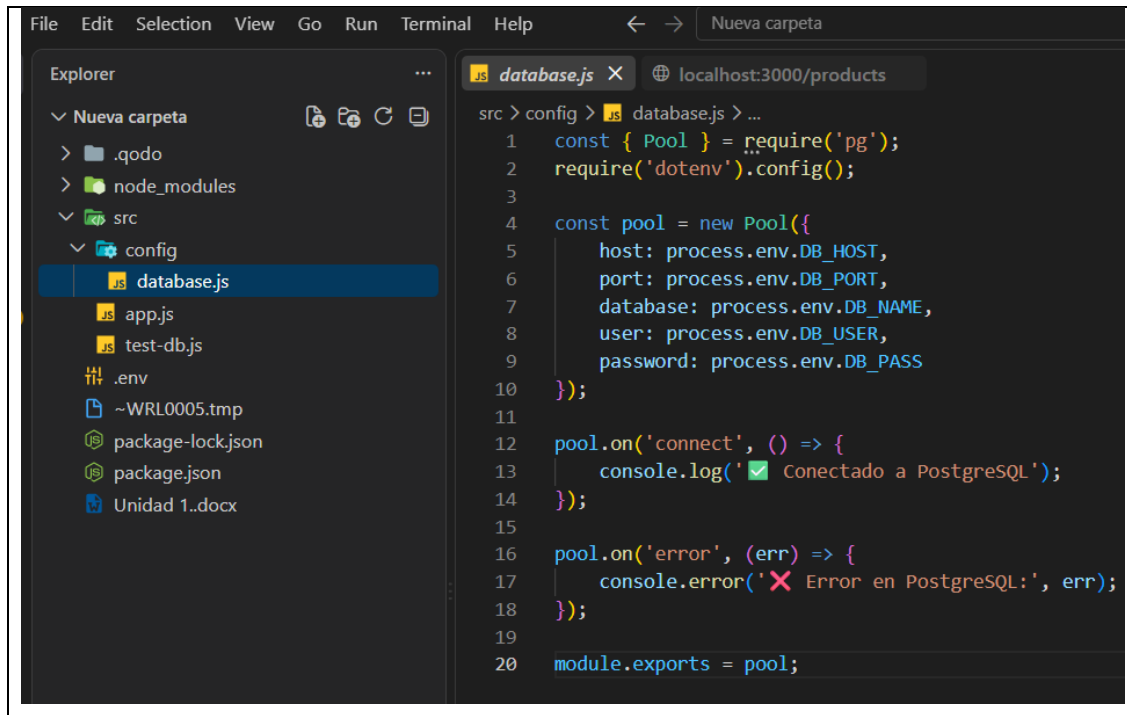


Imagen 4. Configuración de las variables de entorno. VS Code. Fuente: Propia

El parámetro DB_HOST establece que PostgreSQL se encuentra ejecutándose localmente en el equipo de desarrollo, mientras que DB_PORT corresponde al puerto utilizado por PostgreSQL para aceptar conexiones. Por su parte, DB_NAME identifica la base de datos EcoHome, DB_USER corresponde al usuario configurado en PostgreSQL y DB_PASS contiene la contraseña de acceso. Finalmente, la variable PORT establece el puerto 3000 para la ejecución del servidor Express.

Para permitir que Node.js pudiera leer estas variables, se instaló y configuró la dependencia dotenv. Posteriormente, se creó el módulo database.js, encargado de establecer la conexión con PostgreSQL mediante la biblioteca pg y un objeto Pool, lo cual permite administrar las conexiones entre el backend y el sistema gestor de base de datos.

El archivo de configuración implementado fue:



The image shows a screenshot of the Visual Studio Code (VS Code) interface. On the left, the Explorer sidebar displays the project structure. The 'config' folder is expanded, showing files including 'database.js', 'app.js', 'test-db.js', '.env', '~WRL0005.tmp', 'package-lock.json', 'package.json', and 'Unidad 1..docx'. The 'database.js' file is selected. The main Editor window on the right displays the code for 'database.js'. The code uses the 'pg' library and 'dotenv' to establish a database connection pool using environment variables. The code is as follows:

```
src > config > database.js > ...
1  const { Pool } = require('pg');
2  require('dotenv').config();
3
4  const pool = new Pool({
5    host: process.env.DB_HOST,
6    port: process.env.DB_PORT,
7    database: process.env.DB_NAME,
8    user: process.env.DB_USER,
9    password: process.env.DB_PASS
10 });
11
12 pool.on('connect', () => {
13   console.log('✅ Conectado a PostgreSQL');
14 });
15
16 pool.on('error', (err) => {
17   console.error('❌ Error en PostgreSQL:', err);
18 });
19
20 module.exports = pool;
```

Imagen 5. Módulo database.js. VS Code. Fuente: Propia

Esta configuración permitió centralizar el acceso a PostgreSQL y reutilizar la conexión desde los diferentes componentes del backend. Además, se evitó incluir directamente las credenciales dentro del código fuente, lo cual representa una práctica adecuada para la gestión de información sensible y facilita posteriormente la configuración del sistema en diferentes ambientes, como desarrollo, pruebas y producción.

3. Prueba de persistencia

Una vez configuradas las variables de entorno y el módulo de conexión, se creó temporalmente el archivo test-db.js para comprobar que el backend pudiera comunicarse correctamente con PostgreSQL.

La prueba ejecutó una consulta sencilla mediante:

```
C:\Users\juanc\Desktop\Especializacion U Asturias\Materias\Nueva carpeta>node src/test-db.js
◇ injected env (6) from .env // tip: ~ auth for agents [www.vestauth.com]
✓ Conectado a PostgreSQL
✓ Conexión exitosa con PostgreSQL
Fecha del servidor: { now: 2026-08-06T22:23:01.144Z }
```

Imagen 6. Prueba de comunicación. VS Code. Fuente: Propia

La prueba permitió verificar que los cuatro productos previamente registrados en PostgreSQL podían ser consultados desde el endpoint:



Imagen 7. endpoint. VS Code. Fuente: Propia

Como resultado, los registros almacenados en la tabla products fueron retornados correctamente por la API. Esto demuestra que el backend dejó de depender de estructuras de datos almacenadas únicamente en memoria y comenzó a utilizar PostgreSQL como fuente persistente de información para el catálogo.

4. Prueba de persistencia

Lo primero es apagar el servidor:

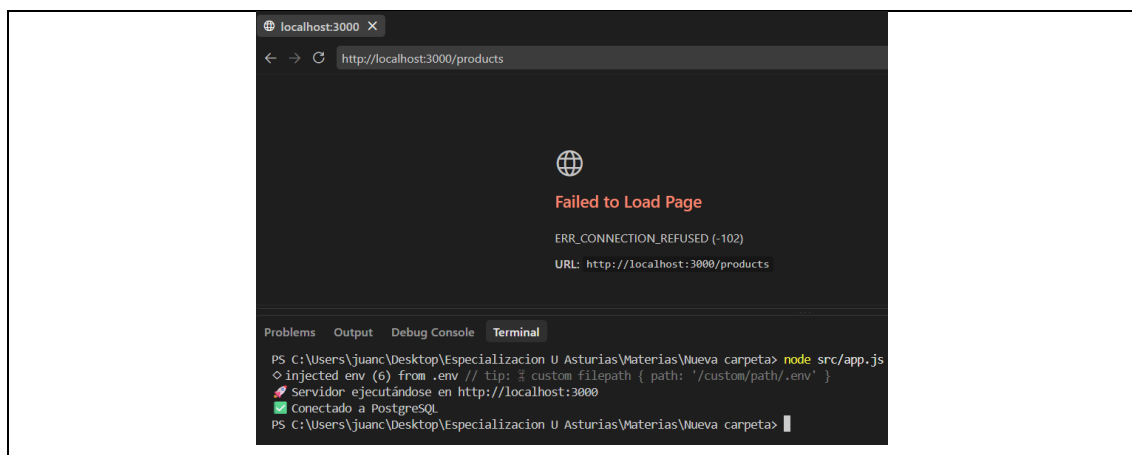


Imagen 8. Apagado del servidor. VS Code. Fuente: Propia

Ahora, se debe encender nuevamente el servidor y volver a consultar en el navegador:

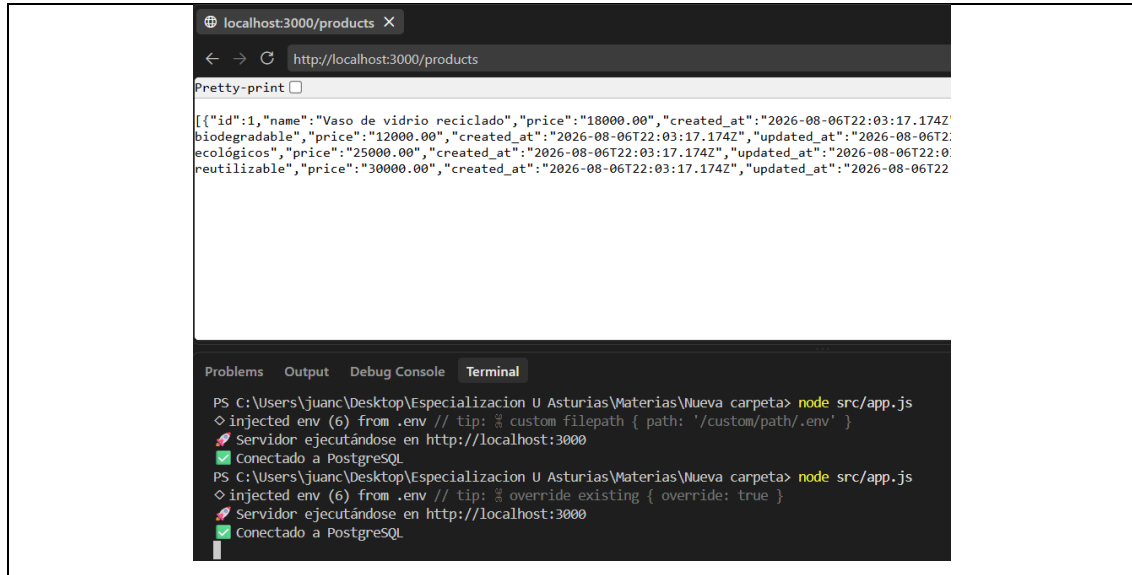


Imagen 9. Encendido del servidor. VS Code. Fuente: Propia

Actividad 2. Autenticación y autorización con JWT

El objetivo consiste en implementar un mecanismo de autenticación que permita registrar usuarios, iniciar sesión mediante credenciales y generar un **JSON Web Token (JWT)**. Posteriormente, el token será utilizado para proteger las operaciones de administración del catálogo.

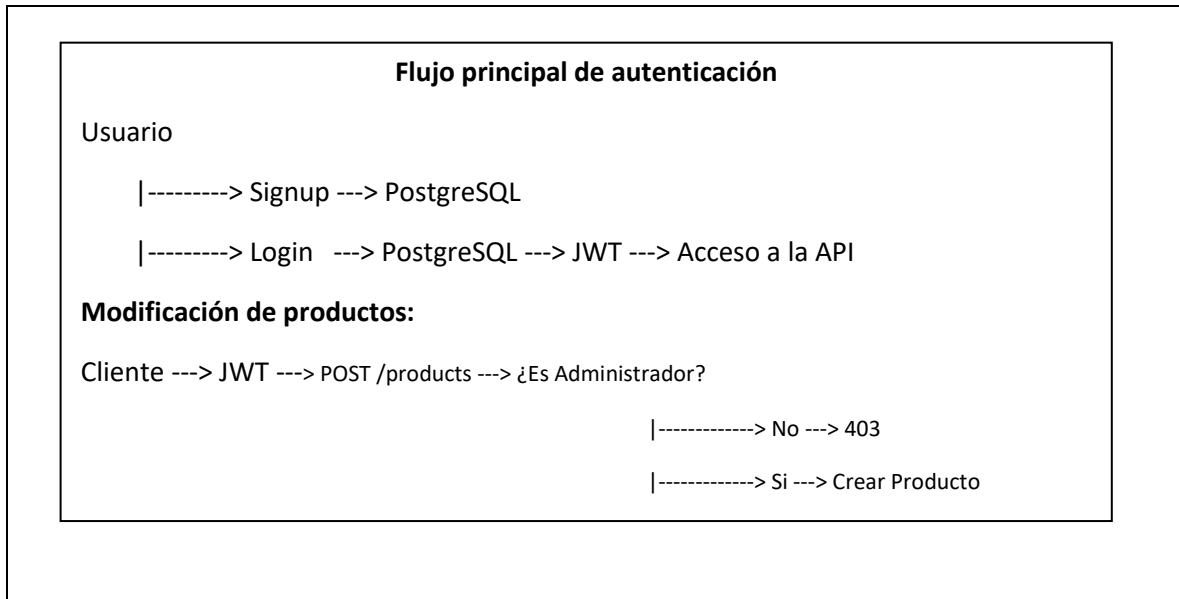


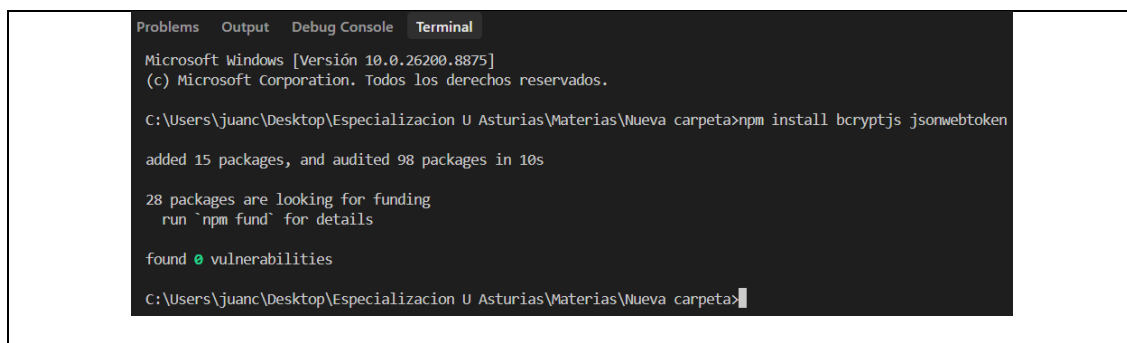
Imagen 10. Flujo de trabajo. Fuente: Propia.

1. Endpoints de autenticación

Anteriormente, se creó la tabla users en la base de datos EcoHome, definiendo los campos necesarios para la identificación y autenticación de los usuarios. El campo email fue configurado como único para evitar registros duplicados y la contraseña se almacenará mediante un valor hash en el campo password_hash, evitando el almacenamiento de credenciales en texto plano.

1.2. Instalación de herramientas para autenticación

En la terminal **CMD** de VS Code ejecuta:



```
Problems Output Debug Console Terminal
Microsoft Windows [Versión 10.0.26200.8875]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\juanc\Desktop\Especializacion U Asturias\Materias\Nueva carpeta>npm install bcryptjs jsonwebtoken

added 15 packages, and audited 98 packages in 10s

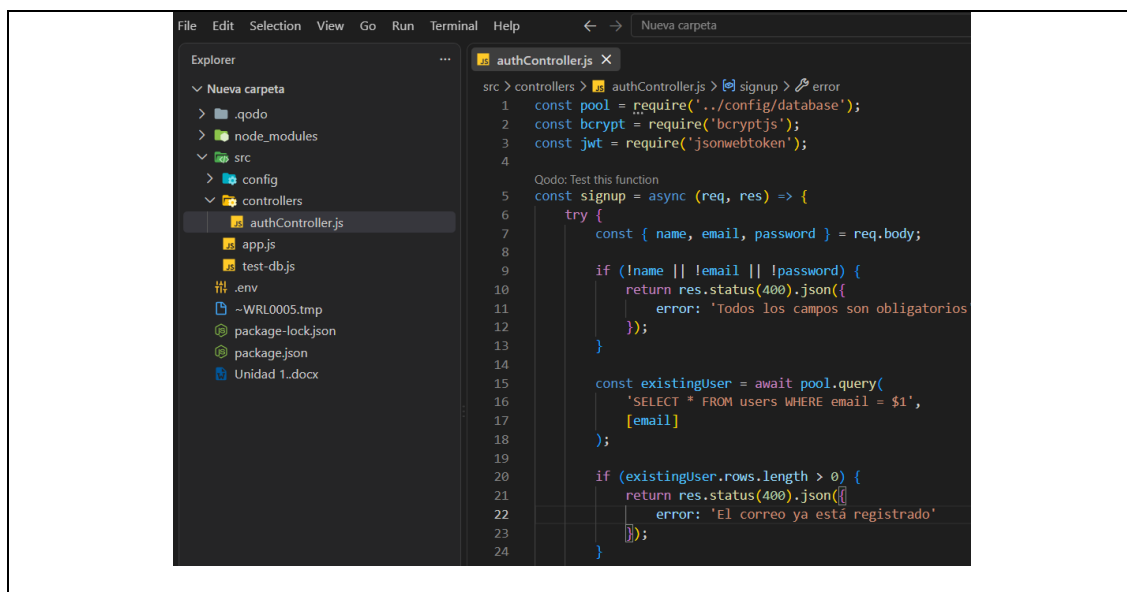
28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\juanc\Desktop\Especializacion U Asturias\Materias\Nueva carpeta>
```

Imagen 11. librerías de autenticación. Fuente: Propia.

1.3. Creación del controlador de autenticación



```
File Edit Selection View Go Run Terminal Help
Nueva carpeta

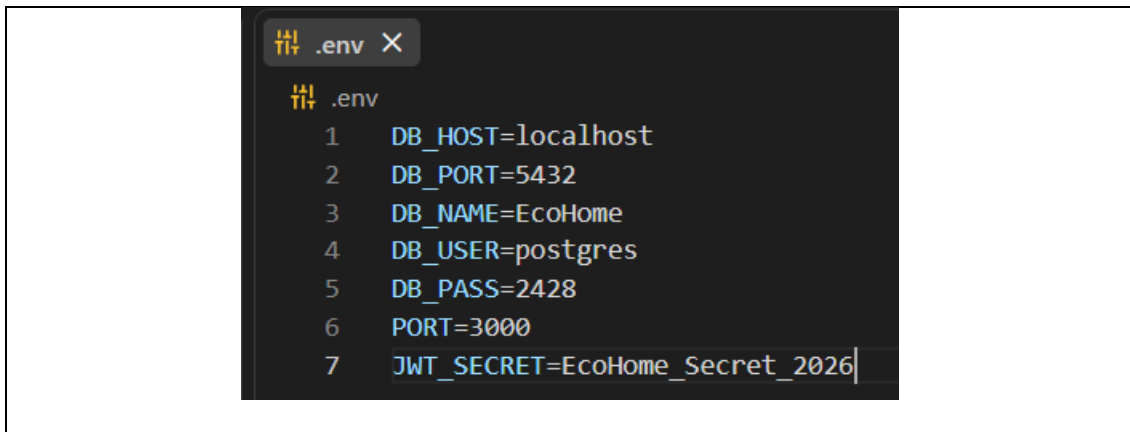
Explorer
  Nueva carpeta
    .qodo
    node_modules
    src
    config
    controllers
      authController.js
      app.js
      test-db.js
    .env
    ~WRL0005.tmp
    package-lock.json
    package.json
    Unidad 1.docx

authController.js
src > controllers > authController.js > signup > error
1  const pool = require('../config/database');
2  const bcrypt = require('bcryptjs');
3  const jwt = require('jsonwebtoken');
4
Qodo: Test this function
5  const signup = async (req, res) => {
6    try {
7      const { name, email, password } = req.body;
8
9      if (!name || !email || !password) {
10         return res.status(400).json({
11           error: 'Todos los campos son obligatorios'
12         });
13       }
14
15       const existingUser = await pool.query(
16         'SELECT * FROM users WHERE email = $1',
17         [email]
18       );
19
20       if (existingUser.rows.length > 0) {
21         return res.status(400).json({
22           error: 'El correo ya está registrado'
23         });
24       }
25     }
26   }
```

Imagen 12. Script “authController.js”. Fuente: Propia.

1.4. Agregar el secreto JWT al .env

Esto se hace en VS Code, dentro del archivo .env que ya creo.



```
.env X
.env
1 DB_HOST=localhost
2 DB_PORT=5432
3 DB_NAME=EcoHome
4 DB_USER=postgres
5 DB_PASS=2428
6 PORT=3000
7 JWT_SECRET=EcoHome_Secret_2026|
```

Imagen 13. Secreto JWT. Fuente: Propia.

1.5. Rutas de autenticación



```
JS authRoutes.js X
src > routes > JS authRoutes.js > ...
1 const express = require('express');
2
3 const router = express.Router();
4
5 const {
6   signup,
7   login
8 } = require('../controllers/authController');
9
10 router.post('/signup', signup);
11 router.post('/login', login);
12
13 module.exports = router;|
```

Imagen 14. Rutas de autenticación. Fuente: Propia.

1.6. Probar POST /auth/signup

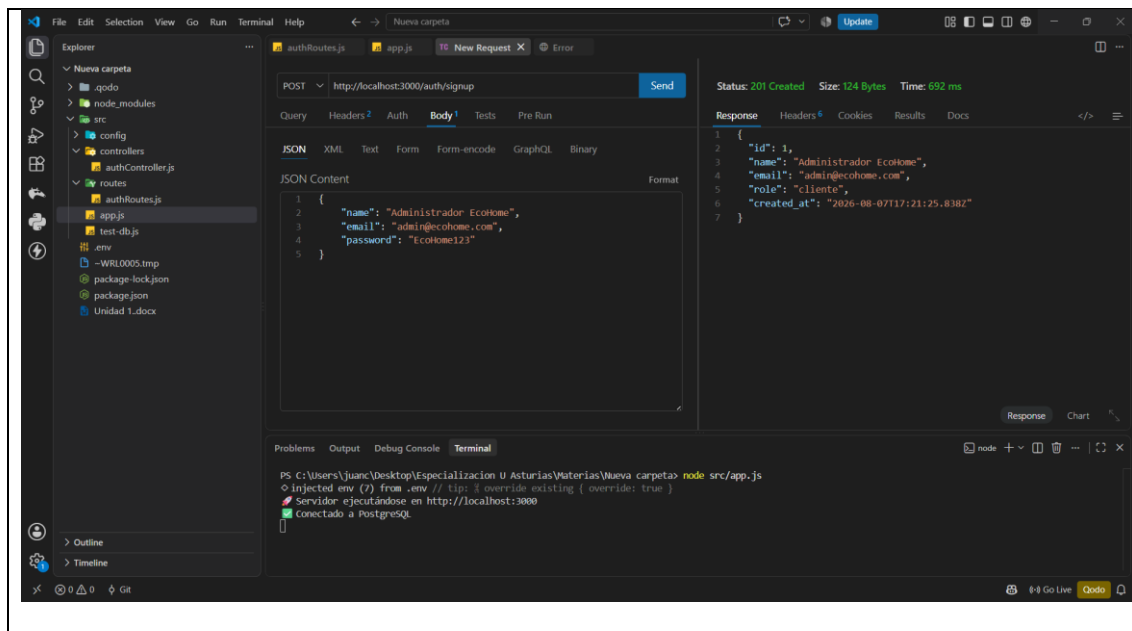


Imagen 15. Thunder Client. Fuente: Propia.

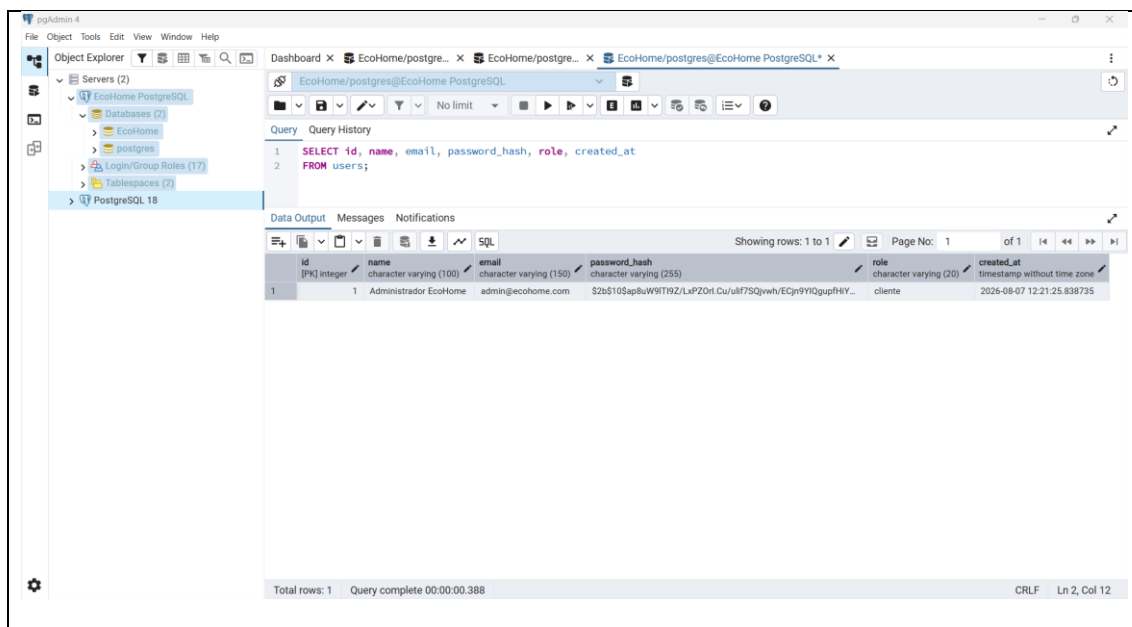
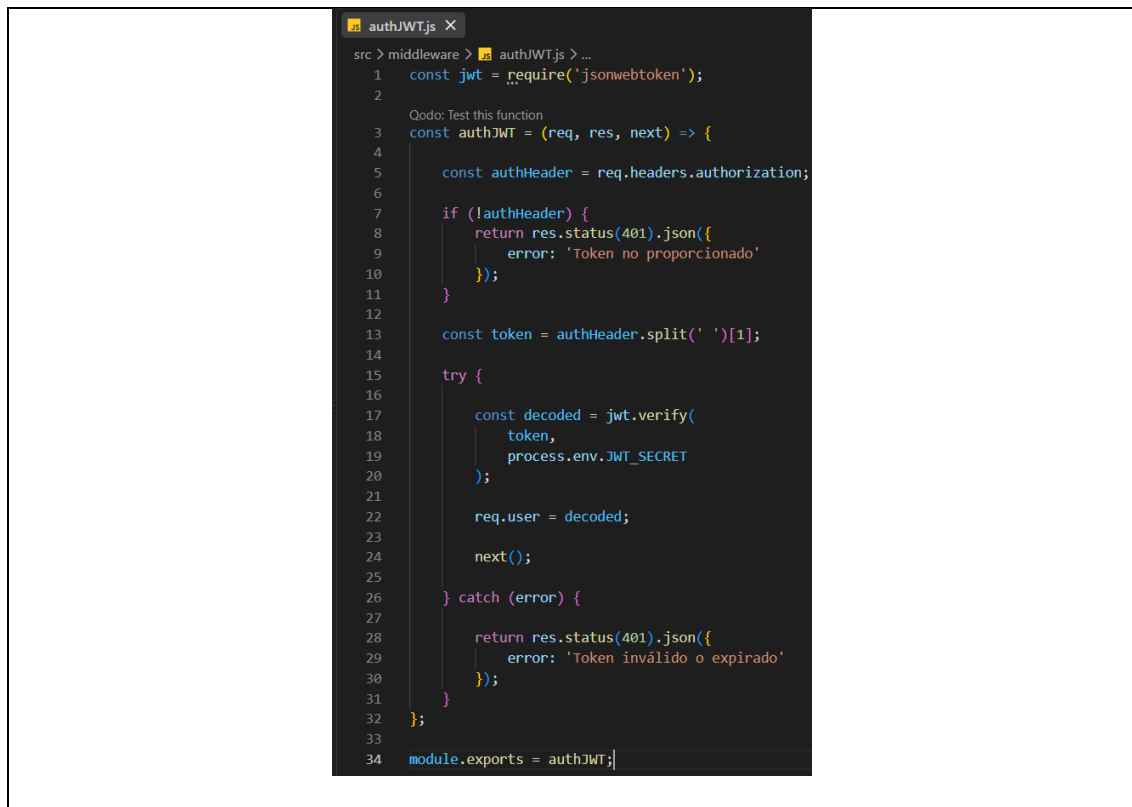


Imagen 16. pgAdmin. Fuente: Propia.

Se realizó una solicitud POST /auth/signup mediante la cual se registró correctamente un nuevo usuario en la plataforma. La contraseña recibida por el backend no se almacena directamente en la base de datos, sino que es procesada mediante bcrypt antes de ser almacenada en el campo

El middleware de seguridad implementado en el sistema EcoHome tiene como finalidad controlar y proteger las solicitudes que llegan al backend antes de que sean procesadas por los diferentes recursos de la aplicación. Este mecanismo permite establecer una capa intermedia encargada de verificar las credenciales de los usuarios y determinar si cuentan con los permisos necesarios para acceder a los endpoints protegidos.



```
1  const jwt = require('jsonwebtoken');
2
3  // Oodo: Test this function
4  const authJWT = (req, res, next) => {
5
6      const authHeader = req.headers.authorization;
7
8      if (!authHeader) {
9          return res.status(401).json({
10             error: 'Token no proporcionado'
11         });
12     }
13
14     const token = authHeader.split(' ')[1];
15
16     try {
17         const decoded = jwt.verify(
18             token,
19             process.env.JWT_SECRET
20         );
21
22         req.user = decoded;
23
24         next();
25     } catch (error) {
26         return res.status(401).json({
27             error: 'Token inválido o expirado'
28         });
29     }
30 }
31
32 module.exports = authJWT;
```

Imagen 18. Script “authJWT.js”. Fuente: Propia.

2.1. Prueba de funcionamiento middleware

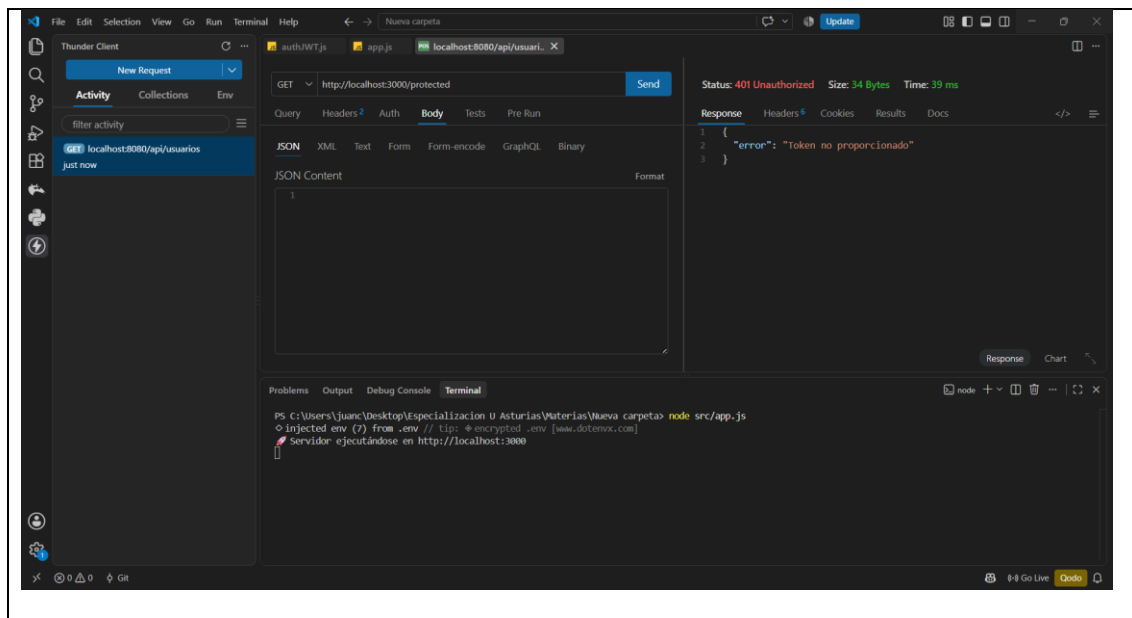


Imagen 19. Prueba de seguridad. Fuente: Propia.

JWT obtenido en el login:

"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJhZG1pbkBIY29ob21lLnVbSIsInJvbnUiOiJjbGllbnRliiwiaWF0IjoxNzg2MTI0MTczLCJleHAiOiJlODYxMzEzNzN9.rlQDu9A501qjGAKvaofIKc65B0QnTQQoJM0F1nnAlyA"

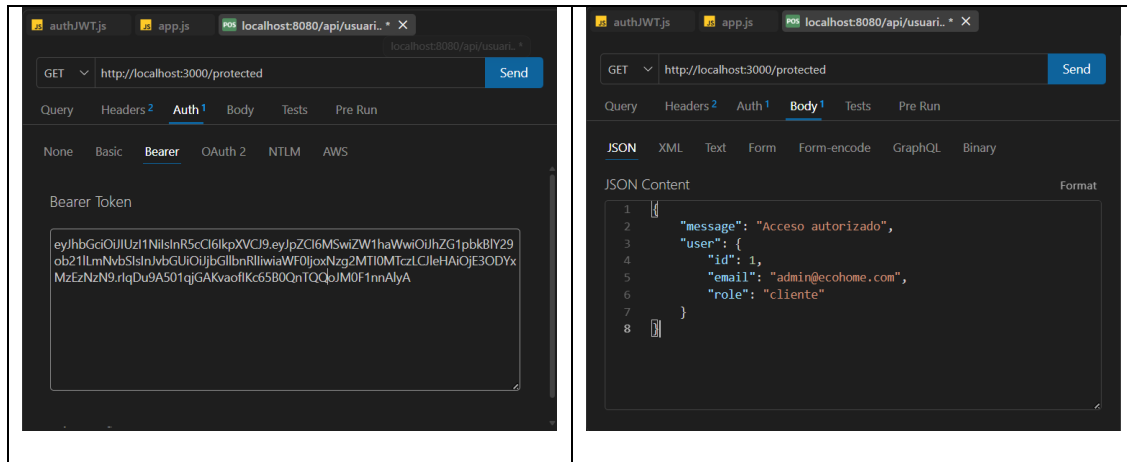


Imagen 20. Authorization → Bearer Token. Fuente: Propia.

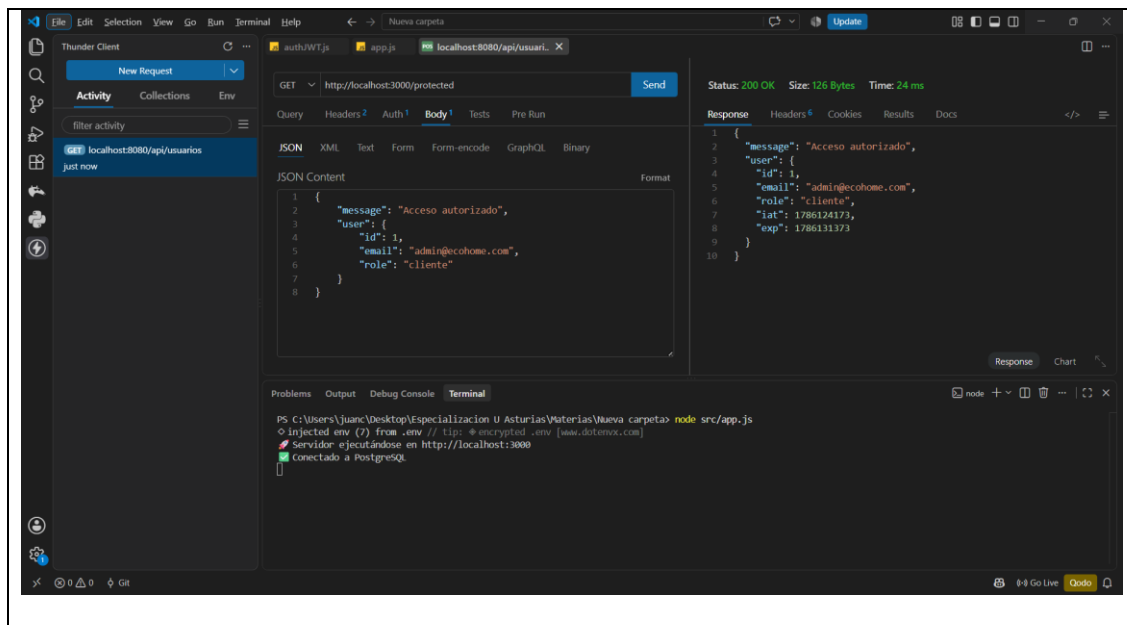


Imagen 21. Authorization → Respuesta. Fuente: Propia.

Se verificó el funcionamiento del middleware authJWT, encargado de validar el token enviado mediante el encabezado Authorization: Bearer. Inicialmente se realizó una solicitud sin token, obteniendo una respuesta HTTP 401, lo que demuestra que las rutas protegidas no pueden ser

Imagen 23. Configuración de nuevo usuario con el role = cliente. Fuente: Propia.

JWT del nuevo usuario:

*"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NCwiZW1haWwiOiJibGllbnRIQGVjb2hv
bWUuY29tIiwicm9sZSI6ImNsaWVudGUiLCJpYXQiOiJlODYxMjU2NDgsImV4cCI6MTc4NjE
zMjg0OH0.s6gILx8Y-ZNZldHVl2BHTh_nf8TF9ZZGzaJjx3ffYDA"*

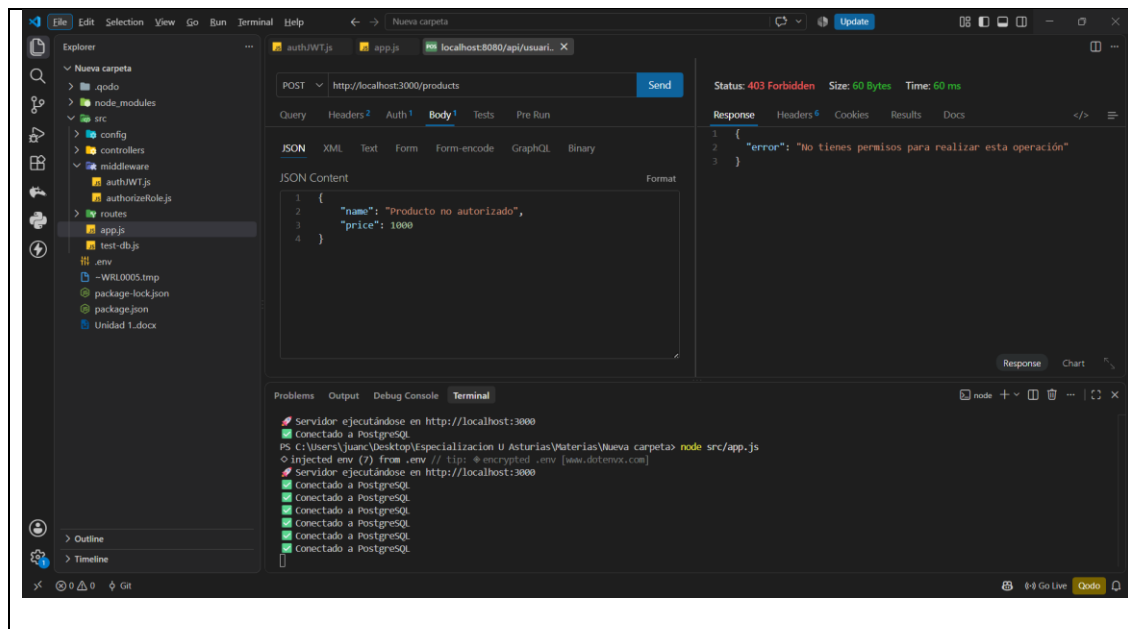


Imagen 24. Respuesta. Fuente: Propia.

Se implementó una separación entre los roles admin y cliente. El usuario administrador posee permisos para gestionar el catálogo, mientras que el usuario cliente únicamente puede realizar operaciones permitidas para usuarios autenticados. Durante la prueba se utilizó un JWT perteneciente a un usuario con rol cliente para intentar crear un producto. El backend rechazó la operación mediante una respuesta HTTP 403 (Forbidden), demostrando que la autenticación por sí sola no concede permisos administrativos. Posteriormente, la misma operación fue realizada con un usuario cuyo rol corresponde a admin, obteniendo una respuesta exitosa.

Aplicación Práctica del Conocimiento

El desarrollo de la solución propuesta para el caso de **EcoHome Store** permitió aplicar de manera práctica los conocimientos adquiridos durante la asignatura relacionados con el desarrollo de software, la arquitectura backend, la gestión de bases de datos y la seguridad de aplicaciones. La evolución del prototipo permitió comprender la importancia de pasar de una solución temporal basada en memoria a una arquitectura que utilice persistencia real mediante PostgreSQL, garantizando que la información del catálogo permanezca disponible incluso después de reiniciar el servidor.

Asimismo, la implementación de los mecanismos de autenticación y autorización permitió aplicar conceptos de seguridad fundamentales en el desarrollo de aplicaciones modernas. El uso de contraseñas almacenadas mediante hash, la generación de tokens JWT y la diferenciación de permisos entre usuarios administradores y clientes permiten controlar quién puede acceder al sistema y qué operaciones puede realizar. Esto resulta especialmente importante en aplicaciones que manejan información comercial, ya que evita que cualquier usuario pueda modificar datos sensibles como los precios o el estado de los productos.

Los conocimientos adquiridos también pueden aplicarse directamente a proyectos profesionales y académicos que requieran desarrollar APIs para aplicaciones web o móviles. La separación entre rutas, controladores, middleware y acceso a datos facilita el mantenimiento del sistema y permite que posteriormente puedan incorporarse nuevas funcionalidades sin modificar completamente la aplicación. De esta manera, la solución desarrollada para EcoHome Store representa una experiencia práctica que permite relacionar los conceptos estudiados con una situación empresarial real, fortaleciendo las competencias necesarias para diseñar sistemas backend más seguros, persistentes, organizados y preparados para futuras ampliaciones.

Referencias

- [1] Flanagan, D. (2020). *JavaScript: The definitive guide* (7th ed.). O'Reilly Media.
- [2] PostgreSQL Global Development Group. (2025). *PostgreSQL 18 documentation*. PostgreSQL.
[PostgreSQL 18 Documentation](#)
- [3] OpenJS Foundation. (2025). *Express 5.x API reference*. Express.js. [Express.js documentation](#)